

Index

Presentation av verktyg: .NET och Azure	1
0. Inledning	1
1. Översikt av verktygsval och motivering till valen	2
2. .NET och Blazor Server (mjukvarukod, backend och webfront).....	2
2.1 .NET	2
2.2 Blazor Server	3
3. Azure och CosmosDB (molntjänster och databas).....	3
3.1 Azure Service Bus	3
3.2 Azure Functions.....	4
3.3 Azure Application Insights	4
3.4 Azure App Service	4
3.5 Azure App Configuration	4
3.6 Azure Key Vault och roll-baserad åtkomst (Managed Identity + RBAC).	4
3.7 CosmosDB serverless	4
4. GitHub, GHCR och Docker (IaC).....	5
4.1 GitHub och GHCR (GitHub Container Registry).....	5
4.2 Docker och containrar	5
5. Huvudfunktionen – Den Stora Röda Knappen	5
6. Sammanfattning och samspel	5
X. Referenser	6

Presentation av verktyg: .NET och Azure

0. Inledning

Fullständigt repo för projektet kan hittas här:

<https://github.com/mymh13/clo24-nikhal78-examwork>

Den har en rätt extensiv dokumentation som återfinns i foldern docs. En hel del är teknisk presentation på engelska, men det finns en del mer lättviktigt material som förklarar syfte och kontext. Kan rekommendera följande:

[Glossary](#) – Förklarar begrepp, förkortningar och termer som används i projektet.

[Systemöversikt](#) – Ger en vid överblick av tanken bakom systemet innan det började byggas.

1. Översikt av verktygsval och motivering till valen

Kunden har idag en applikation där ett äldre och ett nyare system fungerar sida vid sida. Vissa verktyg har refaktorerats och integrerats i det nya verktyget, medan andra lever kvar i det äldre. På grund av det så arbetar kunden i dagsläget i två system, samtidigt.

Målet med detta projekt är att skapa ett gränssnitt för att kunna hantera båda systemen inom samma flöde, den tekniska lösningen blir ett verktyg för att successivt kunna flytta över funktionalitet från det gamla till det nya systemet.

För att inte behöva bygga två fullständiga parallella system, så väljer jag att simulera den här refaktoreringen och hanteringen genom att bygga ett system som kan byta mellan de två systemen genom en toggle switch.

Som huvudsakliga verktyg har jag valt .NET som helhetslösning för koden, CosmosDB som databas och Azure för molnlösningarna. Skälet till val av verktyg är helt enkelt att det är de verktyg som kunden använder redan idag.

2. .NET och Blazor Server (mjukvarukod, backend och webfront)

Verktyget är framför allt till för utvecklare som i dagsläget använder båda systemen samtidigt. Användare av kundens applikation ser inte vilket av systemen de använder, men utvecklare har en miljö där de byter mellan systemen.

Tanken är att utvecklarna skall kunna använda detta projekts verktyg genom att knyta ihop de två systemen, kunna refaktorera kod från det gamla systemet och successivt föra över funktionalitet till det nya, utan att behöva ta ner huvudapplikationen.

Verktygen blir även en backup-lösning som kan styra om trafiken till det ena eller det andra systemet, om problem uppstår vid uppgradering/refaktorering.

2.1 .NET

.NET är en helhetslösning i en utvecklingsmiljö där programspråket är C# (uttalas C-sharp). Inom den miljön finns olika verktyg och lösningar för att knyta ihop alla olika element i en modern applikation. I detta projekt så byggs flera separata enheter som har sina egna funktioner och roller:

- /api/Ticketing.Api - Är själva grundstommen som
- /functions/Ticketing.Functions – Skall förklaras
- /shared/Ticketing.Contracts – Skall förklaras
- /tests/Ticketing.Web.Tests – Skall förklaras

2.2 Blazor Server

Blazor Server hanterar webfronten, det du ser på själva hemsidan. Blazor är baserat på Razor, vilket ger oss möjligheten att skriva C#-kod tillsammans med HTML-kod. Vi kan helt enkelt hålla en konsekven och enhetlig utvecklingsmiljö.

- /web/Ticketing.Web – Hemsida-delen av projektet

3. Azure och CosmosDB (molntjänster och databas)

En liten summering av hur jag presenterar Azure-lösningarna i githubrepot:

Service	Purpose	Tier/Plan
App Service	Run Blazor Server and API application in the same App Service plan.	Basic B1 (dev)
App Configuration	Centralized management of settings and feature flags between environments. Hot-reload via sentinel key pattern.	Free (dev)
Key Vault	Secure storage of secrets and connection strings. Managed identity access via RBAC.	Standard
Application Insights	Logging, monitoring, and performance measurement via KQL. Used by both App Service and Function App. Custom events track dual-system architecture flows.	Pay-as-you-go
API Management (APIM)	Gateway for public GET endpoints, caching, and rate limiting.	(Future)
Cosmos DB	Serverless NoSQL database for bookings, users, and outbox events. Partition key strategy for efficient querying.	Serverless
Service Bus	Message queue for event-driven architecture. <code>booking-events</code> queue with dead letter queue support.	Basic (dev)
Azure Functions	Serverless event processing. Processes Service Bus messages for event-driven flow.	Basic B1 (dev)
Storage Account	Required for Azure Functions runtime and state management.	Standard LRS

Tjänsterna ligger till vänster, syftet beskrivs i mitten och tier/plan är vilken kostnadsnivå eller implementations-variant som används. Förklarar de olika funktionerna lite mer ingående:

3.1 Azure Service Bus

Information som går igenom event-flödet behöver hanteras. Tänk dig att du skall kliva på ett tåg, resenärer kommer vid olika tillfällen men ni står och väntar, i en kö, vid perrongen. När tåget är redo att transportera er, då kliver ni på i den ordningen som ni kom till perrongen. Blir tåget fullt så får ni vänta på nästa.

På samma sätt så hanterar Service Bus de event som kommer in i vårt system. All internettrafik sker inte omgående, det kan vara en viss fördröjning, och att använda en tjänst som Service Bus öppnar upp för möjligheter att hantera data på ett annat sätt än om du inte hade tjänsten: exempelvis skulle du kunna säga att du inte ställer ut en biljett till en resenär om inte betalningen kommit in (nu finns inte just den specifika tjänsten i det här systemet, men teoretiskt skulle det kunna appliceras).

Service Bus är alltså själva kö-systemet eller koordinatören som styr om resenärerna.

3.2 Azure Functions

Det här är själva tågföraren som kör din trafik som kommit in från Service Bus. När koordinatören ger ok, eller kön kommit fram, då är det dags att köra iväg med (processa) dina resenärer. Functions ser till att din resenärer (informationen) förflyttas.

3.3 Azure Application Insights

Som namnet antyder så ger Insights just insikt och en överblick av din applikation. Både App Service och Functions använder Insights för att visa loggar och realtidsdata.

För demonstrationssyfte har jag byggt en Workbook i Insights, där jag kan visa på hur dataflödet ser ut beroende på om applikationen kör synkroniserat eller event-baserat. Det ger ett tydligt värde för användaren som kan se en direkt effekt av Toggle-knappen.

För att återgå till tåg-allegorin så kan vi tänka oss att Insights visar i realtid hur dina tåg rör sig, hur växlarna till de olika spåren står kopplade, hur många vagnar ett tåg har, etc.

3.4 Azure App Service

App Service kör Blazor Server och API-applikationen. Vad skiljer den mot en "server" (virtuell maskin)? Jo, **bla bla bla**

Hur passar den in i tåg-allegorin?

3.5 Azure App Configuration

Det här är en nyckelfunktion som vi använder för att styra mellan de eventbaserade och de synkroniserade tjänsterna. Det är en konfigurations-hanterare.

Säg att du kikar i Insights och ser att ditt tåg är på väg mot en växel som kommer styra in tåget på det vänstra spåret, men du vill ha det högra. Du trycker på Toggle-knappen, och då har du redan förutbestämda inställningar för det i Configurations. Du kan alltså ha olika flaggor i ditt system som vinkar tågen åt de håll du vill, och förutbestämt: du vet att en lila flagga betyder att tåget skall köra i 62 km/h och alltid ta vänster vid alla spårval. Exempelvis.

3.6 Azure Key Vault och roll-baserad åtkomst (Managed Identity + RBAC).

I Key Vault lagras data som vi vill säkra från åtkomst, lösenord, anslutningsinformation. Det skyddar datan mot att exponeras när vi transporterar data över publika nät, exempelvis. Vi använder även Managed Identity

3.7 CosmosDB serverless

CosmosDB är vår databas som hanterar lagrad information. Vilka biljetter har vi ställt ut? QR-koder har genererats? Vilka användare/vilken användarinformation? Har vi event som går via en outbox så går trafiken via outboxen istället för rakt in i inboxen.

4. GitHub, GHCR och Docker (IaC)

För att få koden från utvecklarens dator till tjänsterna på molnet så använder det här projektet framförallt GitHub, med GitHub Container Registry, samt Docker.

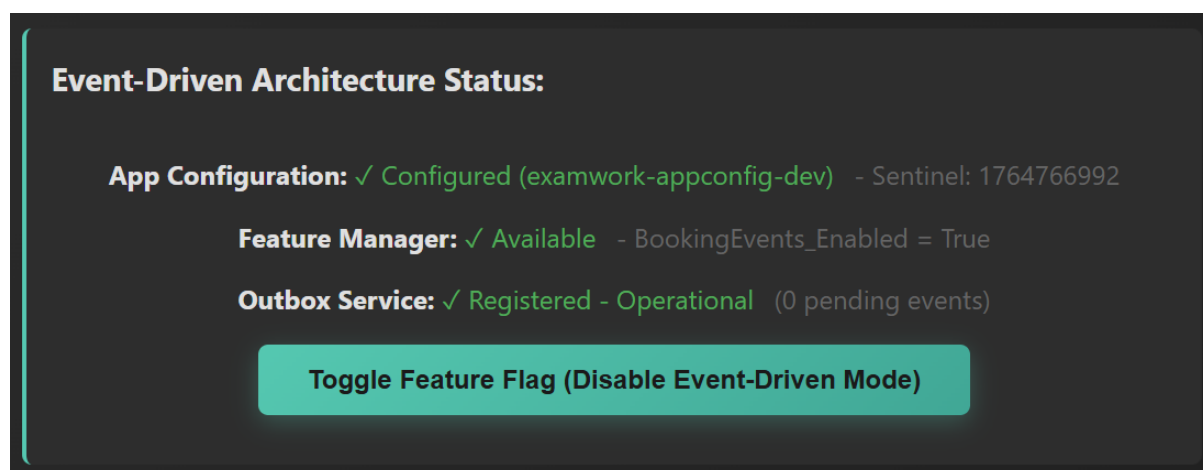
4.1 GitHub och GHCR (GitHub Container Registry)

GitHub är både versionshanterare av mjukvaran, och även transportverktyget som flyttar koden. Koden laddas up i en container till GitHubs Container Registry, vilket är en lagringsplats för containrar. En container är en avgränsad "låda" som innehåller den kod du väljer att lägga på den.

4.2 Docker och containrar

För att stoppa vår kod i containrar så använder vi Docker. Det är ett system med diverse hjälpmedel för att just containerizera och transportera kod. När vår kod är redo att flyttas från utvecklarplattformen till molnet, då kör vi en byggprocess av koden lokalt och sedan packas den in i containern, redo att skeppas.

5. Huvudfunktionen – Den Stora Röda Knappen



Det mest centrala verktyget vi har i den här lösningen är Toggle-knappen som hjälper oss att byta mellan de två systemen. Utan att gräva ner sig för djupt i tekniska detaljer så kan vi helt enkelt säga att det är en knapp som slår av och på hur en biljettbokning styrs. Går det direkt till databasen, eller tar det en omväg via en mellanlagring?

Omkring detta ligger många olika tekniska lösningar för att möjliggöra den nyss beskrivna funktionaliteten, det är multipla beroenden av olika funktioner, och symbiosen mellan dem, som skapar de förutsättningar vi behöver för att kunna implementera "Den Röda Knappen".

6. Sammanfattning och samspel

Flödet mellan de olika delarna kan visualiseras så här:

Infoga här ett flödesschema eller liknande som visar webfront/.NET, APIer/.NET, databas/CosmosDB samt olika transportsträckor: synkron direkt mellan DB och applikation, och en eventbaserad som går via Service Bus, Functions etc.

X. Referenser

Min dokumentation i githubrepo har täckt referensmateriet mer än väl. ADR-sektionen i repots har verktygsspecifika referenser i slutet av varje ADR. ADR står för Architectural Design Records och betyder helt enkelt ett beslutsunderlag med motivering till varför jag gjort de designval jag gjort. Du hittar dem här:

<https://github.com/mymh13/clo24-nikhal78-examwork/tree/main/docs/adr>